

## C++ 교육 1일차 보고서

예비단원 김민찬

### 1. 도입

본 보고서는 C++ 교육 1일차 과정에서 배운 내용을 이해하고 분석한 것, 그 과정에서 느낀 내용과, 이의 활용 방안 등을 정리하여 작성한 것이다. 이번 교육에서는 C++에서의 namespace, 입/출력과 참조자, 객체의 개념과 문법에 대해 배웠다.

### 2. namespace(이름 공간)

C++에는 namespace(이름 공간)라는 개념이 등장한다. 이러한 이름 공간은, 같은 키워드 간 충돌을 막기 위해 도입되었으며, 동일 키워드에 대해, 이름 공간을 거쳐 호출하게 되면, 현재 공간에서 정의된 키워드가 아닌, 호출한 이름 공간에서 정의된 키워드를 호출하게 된다. 예컨대, main과 별개로 정의된 place라는 이름 공간 두 곳에 a라는 변수/함수가 존재할 때, 이름 공간 호출(::)을 통해 a를 호출하면 place의 a가 호출되는 것이다.

### 3. cout/cin

C++에는 C에서와는 달리, printf, scanf를 사용할 수 없으나, 해당 기능에 대응되는 cout과 cin이 존재한다. 두 키워드는 이름 공간 std에 선언된 객체이며, 이를 사용하기 위해선 이름 공간 std에서 호출하여야 한다.

cout: cout은 스트림 삽입 연산자로 기능하는 '<<'을 통해 값을 화면에 출력할 수 있다. 또한, printf와 달리 형식 지정자가 필요 없어, 다양한 자료형과 변수를 '<<' 연산자를 붙여 한 번에 출력할 수 있다. 그리고 "\n"을 사용하여 줄바꿈도 가능하지만, std 공간의 endl을 통해서도 줄바꿈을 실행할 수 있다.

cin: cin은 cout과 달리, 스트림 추출 연산자로 기능하는 '>>'을 통해 입력값을 변수에 저장할 수 있으며, cout과 마찬가지로, 연산자를 통해 여러 값을 한번에 저장할 수도 있다.

### 4. 참조자(reference)

C++에는 C와 동일하게 포인터의 개념이 존재하나, '참조자'라는 새로운 개념이 등장한다. 참조자는 포인터와 같이 변수의 주소에서 값을 읽어올 수 있으나, 포인터와 달리 참조자는 메모리 주소가 아닌 그 값을 가리키며, 변수만을 참조할 수 있다. 또 참조자가 참조하는 변수는 재정의할 수 없으며, 선언 단계에서만 정의할 수 있다. 정리하면 함수를 가리키는 또 하나의 이름을 만드는 것이다.

### 5. Class, 절차 지향과 객체 지향

C와 C++의 차이는 C는 절차 지향 언어인 반면, C++은 객체 지향 언어라는 것이다.

절차 지향 언어에서는 여러 개의 자료를 절차에 따라 연산함으로 프로그램을 제작하였지만, 객체 지향 언어에서는, 절차보다, 각 객체에 따라 다른 연산을 할 수 있으며, 이러한 객체를 모아 프로그램을 제작한다는 차이를 가진다.

이에 따라, C언어에서는 구조체를 통해 변수를 멤버로 가질 수 있었지만, C++에서는 class를 통해 변수 뿐만 아니라 함수 또한 멤버로 가질 수 있게 되었으며 데이터 접근에도 제한을 만들 수 있게 되었다.

이러한 객체 지향 언어에서는 class를 통해 객체를 제작할 수 있으며, C++에서는 다음 방식을 통해 class를 제작한다.

class ‘클래스명’ - 제작할 클래스의 이름이며, 이후 중괄호 안에 클래스의 멤버를 선언할 수 있다.

private/public - private과 public은 클래스의 멤버가 존재하는 공간이며, 외부에서의 참조를 제한할 수 있다. 예를 들어, private 영역에 선언된 멤버 a와 public 영역에 선언된 멤버 b가 있을 때, 객체의 b는 a를 참조할 수 있지만, 외부에서 객체의 a를 호출하는 것은 불가능하다.

‘클래스명’()/~‘클래스명() - 클래스에는 생성자와 소멸자라는 함수가 존재한다. 생성자는 객체를 선언할 때 호출되는 함수이며, 이 함수는 후술할 overloading(동일한 이름의 함수 생성)이 가능

소멸자 함수는 생성자 함수와 달리, 객체가 소멸될 때(일반적으로 프로그램이 종료될 때) 실행되는 함수이고, 클래스에서 오직 하나만 선언할 수 있다.

## 6. class-overloading

클래스에서는 동일 이름의 함수를 여러개 작성하는 것이 가능하다. 단, 이러한 함수를 제작하기 위해선 조건이 존재한다. 먼저, 함수의 이름은 동일하여야 한다. 또, 함수의 인자는 달라야 한다. 여기서 인자가 다른, 인자의 개수가 다르거나, 인자의 자료형이 다른 것을 의미한다.

예를 들어, A라는 함수를 overloading할 때, A(void), A(void)와 같이 인자가 동일하면 오류가 발생하며, A(void), A(int b), A(float c)와 같이 각각 인자가 다른 경우 overloading이 가능하다.

## 7. 느낀 점

이번 교육에서 배운 내용 중 가장 크게 와닿은 것은 class라는 개념의 활용 가능성이다. 객체를 요소로 가지는 객체를 정의하는 방식으로 작성하면 여러 대상에 대해 필요한 요소만을 선택적으로 부여하고, 이를 기반으로 연산을 수행하는 것이 매우 용이해질 것이라는 생각이 들었다. 지금까지 C언어에서는 하나의 자료 구조에 필요한 모

든 변수를 함께 선언하고, 그 자료를 처리하는 함수들을 별도로 흩어놓아 관리해야 했다. 그러나 class를 활용하면, 각 대상이 실제로 필요로 하는 자료와 함수만을 선별하여 하나의 단위로 묶어 정의할 수 있으므로, 대상마다 불필요한 요소를 포함시키지 않으면서도 각각에 최적화된 구조를 만들 수 있다는 점을 깨달았다.

또한 객체를 멤버 변수로 가지는 class를 작성하여 사용한다면, 실제 로봇의 코드를 작성할 때 이를 효과적으로 응용할 수 있을 것이라는 생각이 들었다. 예를 들어 로봇을 구성하는 각각의 기능(적외선 센서, 압력센서, 비전 등)을 독립적인 class로 미리 제작해두고, 실제 로봇 코드를 작성할 때는 해당 로봇에 알맞은 class만을 선택하여 상위 class의 멤버 변수(객체)로 추가하는 방식을 사용할 수 있을 것이다. 이렇게 하면 로봇의 종류나 사양이 달라지더라도, 전체 코드를 처음부터 다시 작성할 필요 없이 필요한 class만 골라서 조합하면 되므로 효율적인 코드 작성이 가능할 것이라 판단하였다.

더 나아가, 만약 기존에 제작해둔 class 중에 해당 로봇에 필요한 요소가 없다면, 그때는 정의되지 않은 요소만을 따로 담은 새로운 class를 제작하여 기존 구조에 추가하는 방식으로 대응할 수 있을 것이라는 생각에 이르렀다. 즉 처음부터 모든 경우의 수를 대비한 class를 만들어 두는 것이 아니라, 필요할 때마다 부족한 부분만을 담당하는 class를 새로 제작하여 끼워 넣는 방식을 사용한다면, 특정 로봇이나 상황에 국한되지 않고 다양한 대상에 범용적으로 적용할 수 있는 코드 구조를 만들 수 있을 것이라고 느꼈다.

## 8. 향후 코드 작성 방향

위에서의 생각을 바탕으로, 앞으로 코드를 작성할 때 다음과 같은 방향을 취해보고자 한다.

1. 하나의 대상을 표현하는 class를 설계할 때, 그 대상이 실제로 필요로 하는 자료와 기능만을 선별하여 포함시키고자 한다. 불필요한 요소까지 모두 담은 class를 만들기보다는, 각 대상의 역할에 맞게 꼭 필요한 멤버만으로 구성된 class를 만들어, 이후 여러 대상에 대해 연산을 수행할 때 각 class가 자신에게 맞는 요소만으로 효율적으로 동작하도록 설계하고자 한다.

2. 1에서의 클래스를 설계하면서, 로봇과 같이 여러 구성 요소로 이루어진 시스템을 코드로 작성할 때는, 전체 기능을 하나의 class가 아니라, 모터·센서·통신 모듈 등을 기능 단위로 개별 class를 제작한 뒤, 상위 class가 이러한 개별 class들을 객체 형태의 멤버 변수로 포함하도록 구성하고자 한다. 이렇게 설계하면 로봇의 종류가 달라지더라도, 해당 로봇에 알맞은 class만 선택하여 상위 class에 추가하는 방식으로 범

용적으로 활용 가능한 코드를 작성할 수 있을 것이라 생각한다.

앞으로, 이러한 코드 방향성을 따라 코드를 작성함으로, 특정 대상에만 종속되지 않고, 다양한 대상에 유연하게 적용할 수 있는 범용적인 코드 구조를 작성해보려 한다.